

UNIVERSIDAD DE



Práctica 1 de Aprendizaje por Refuerzo

José María García Ortiz
Ana Gil Molina
Levi Malest Villarreal

03/03/2025

Índice

- 1. Introducción al problema del bandido de k brazos**
- 2. Resolución mediante algoritmos ε -greedy**
- 3. Resolución mediante algoritmos de ascenso de gradiente**
- 4. Resolución mediante algoritmos UCB**
- 5. Conclusiones de los experimentos**
- 6. Introducción a problemas más complejos**
- 7. Resolución con Monte Carlo On-policy**
- 8. Análisis de resultados y propuesta de mejoras**



Introducción al problema del bandido de k brazos

Introducción al bandido de k brazos

El problema del bandido de k brazos es un marco fundamental en el aprendizaje por refuerzo y la toma de decisiones secuenciales bajo incertidumbre. Inspirado en el dilema de un jugador en un casino que enfrenta múltiples máquinas tragamonedas (cada una con una distribución desconocida de recompensas), este problema captura la tensión entre la exploración y la explotación: ¿debería el jugador seguir eligiendo la máquina que ha dado mejores resultados hasta ahora o probar nuevas máquinas en busca de una recompensa aún mayor?

Introducción al bandido de k brazos

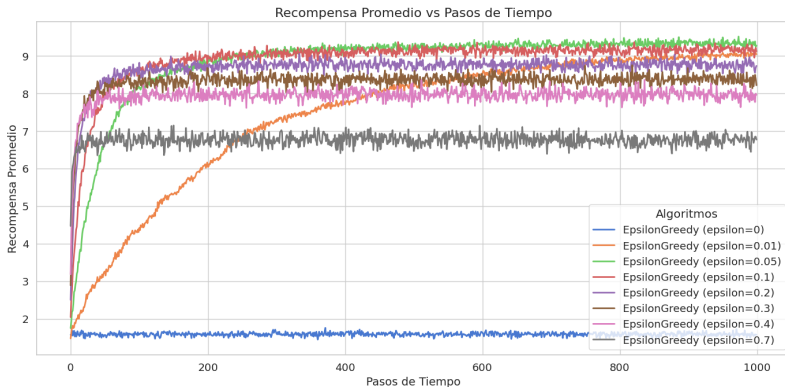
Para abordar este problema, se han desarrollado diversas estrategias que buscan equilibrar exploración y explotación de manera eficiente. Entre ellas, hemos seleccionado 3 familias de algoritmos para un estudio comparativo entre las distintas aproximaciones que ofrecen:

- ▶ **ϵ -greedy**: Alterna entre exploración aleatoria y explotación de la mejor opción conocida con una probabilidad ϵ .
- ▶ **Upper Confidence Bound (UCB)**: Utiliza un criterio de optimismo en la incertidumbre para favorecer brazos con mayor potencial de mejora.
- ▶ **Ascenso de Gradiente**: Ajusta probabilidades de selección mediante una estrategia basada en gradientes para mejorar progresivamente las elecciones del agente.

***Resolución mediante
algoritmos ε -greedy***

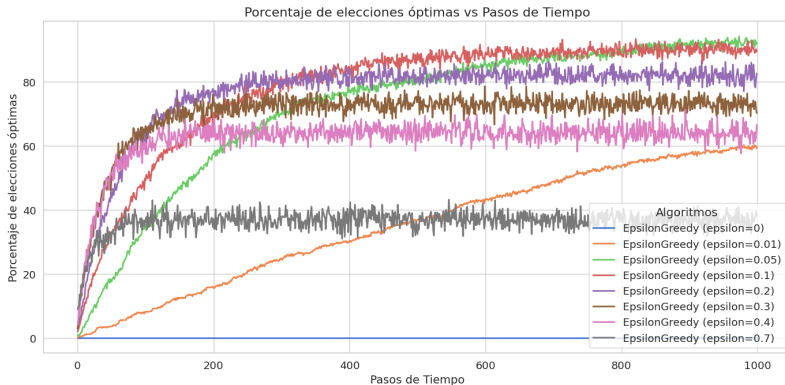
Resolución mediante algoritmos ε -greedy

► Recompensa Promedio frente a Pasos de Tiempo.



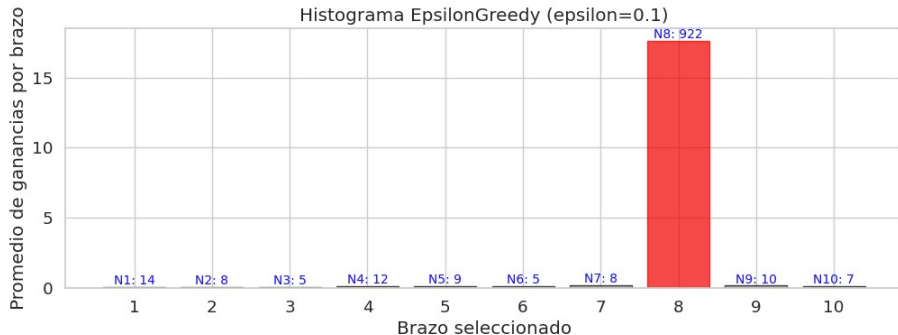
Resolución mediante algoritmos ϵ -greedy

- **Porcentaje de elecciones óptimas frente a Pasos de Tiempo.**



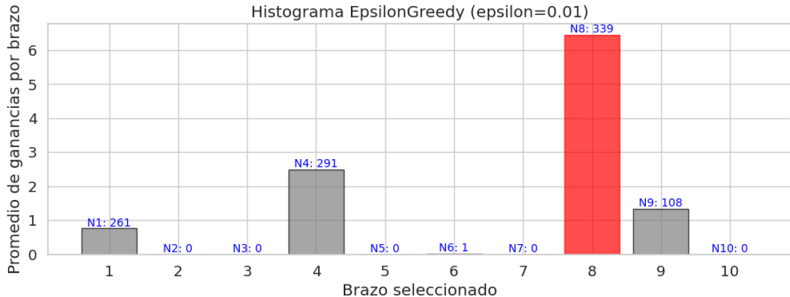
Resolución mediante algoritmos ϵ -greedy

- Estadística por brazo.



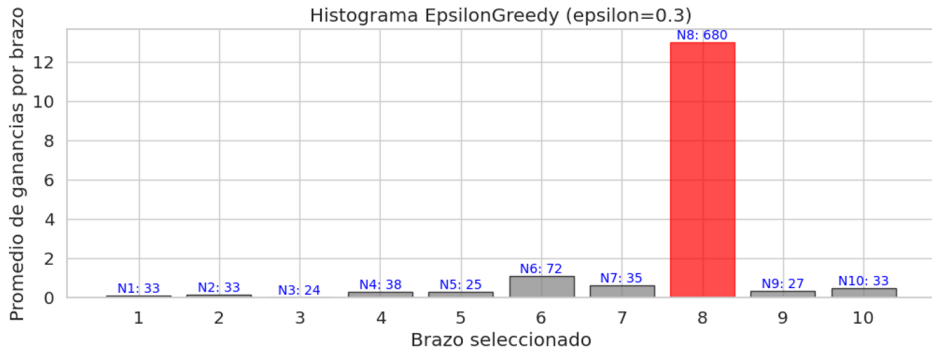
Resolución mediante algoritmos ϵ -greedy

- Estadística por brazo.



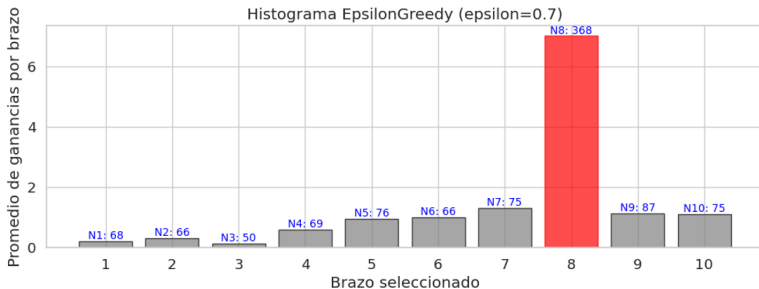
Resolución mediante algoritmos ϵ -greedy

- Estadística por brazo.



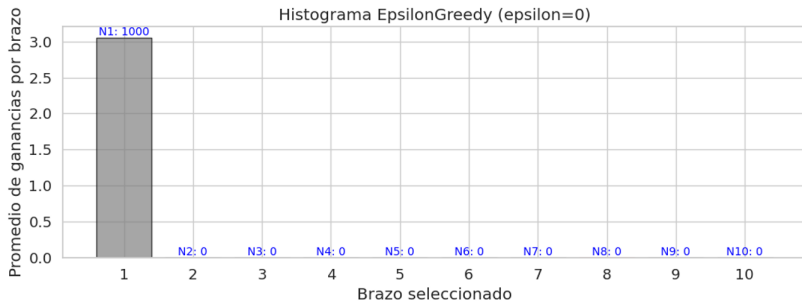
Resolución mediante algoritmos ϵ -greedy

- Estadística por brazo.



Resolución mediante algoritmos ϵ -greedy

- Estadística por brazo.



***Resolución mediante algoritmos
de ascenso de gradiente***

Resolución mediante ascenso de gradiente

Algoritmo Softmax

Variables y notación:

- ▶ Bandido de $K := 10$ brazos $\longrightarrow \mathbb{B} := (a_1, \dots, a_K)$
- ▶ Variable de instante temporal: t , Instante temporal actual: T
- ▶ Variable aleatoria que muestrea un brazo en el instante t : A_t
- ▶ Parámetro de regulación del aprendizaje: τ
- ▶ Política de elección de brazo en el instante t :

$$\pi_t(a_j) \equiv P(A_t = a_j) := \frac{e^{\frac{Q_t(a_j)}{\tau}}}{\sum_{h=1}^K e^{\frac{Q_t(a_h)}{\tau}}} \quad \forall 1 \leq j \leq K$$

- ▶ Variante Adaptativa: $\tau \equiv \tau_t := \frac{\tau_0}{1+\alpha t}$

Resolución mediante ascenso de gradiente

Pasos del algoritmo Softmax:

1. Inicializaciones: $t := 0$, $Q_0(a_j) := 0 \quad \forall 1 \leq j \leq K$, $\tau > 0$

2. Cálculo iterativo:

while $t < T$:

computamos la probabilidad de elección $\pi_t(a_j) \quad \forall 1 \leq k \leq K$

definimos una distribución de probabilidad $\pi_t := (\pi_t(a_1), \dots, \pi_t(a_K))$

A_{t+1} selecciona uno de los K brazos (a) con probabilidad según π_t

$t += 1$

se recibe la recompensa R_t correspondiente a la acción A_t

actualizamos la recompensa esperada del brazo seleccionado:

$$Q_t(a) := Q_{t-1}(a) + \frac{1}{K} (R_t - Q_{t-1}(a))$$

Resolución mediante ascenso de gradiente

Algoritmo de gradiente de preferencias

Variables y notación:

- ▶ Bandido de $K := 10$ brazos $\longrightarrow \mathbb{B} := (a_1, \dots, a_K)$
- ▶ Variable de instante temporal: t , Instante temporal actual: T
- ▶ Variable aleatoria que muestrea un brazo en el instante t : A_t
- ▶ Parámetro de regulación del aprendizaje: α
- ▶ Preferencia asociada a un brazo a en el instante t : $H_t(a)$
- ▶ Recompensa promedio del agente en el instante t : $\bar{R}_t$
- ▶ Política de elección de brazo en el instante t :

$$\pi_t(a_j) \equiv P(A_t = a_j) := \frac{e^{H_t(a_j)}}{\sum_{h=1}^K e^{H_t(a_h)}} \quad \forall 1 \leq j \leq K$$

Resolución mediante ascenso de gradiente

Pasos del algoritmo de gradiente de preferencias:

1. Inicializaciones: $t := 0$, $H_0(a_j) := 0 \quad \forall 1 \leq j \leq K$, $\alpha > 0$

2. Cálculo iterativo:

while $t < T$:

computamos la probabilidad de elección $\pi_t(a_j) \quad \forall 1 \leq k \leq K$

definimos una distribución de probabilidad $\pi_t := (\pi_t(a_1), \dots, \pi_t(a_K))$

A_{t+1} selecciona uno de los K brazos (a) con probabilidad según π_t

$t + 1$

se recibe la recompensa R_t correspondiente a la acción A_t

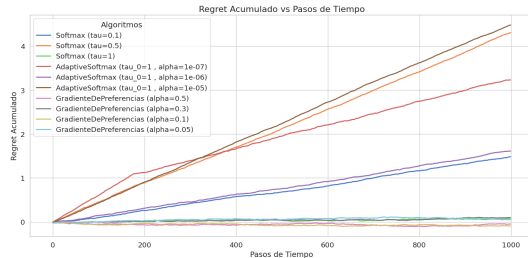
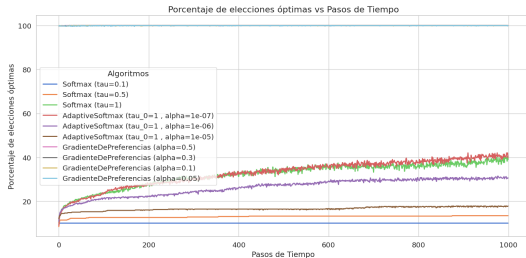
actualizamos las preferencias de todos los brazos según:

$$\begin{aligned} H_t(a) &:= H_{t-1}(a) + \alpha (R_{t-1} - \bar{R}_{t-1}) (1 - \pi_{t-1}(a)) \\ H_t(a_j) &:= H_{t-1}(a_j) - \alpha (R_{t-1} - \bar{R}_{t-1}) \pi_{t-1}(a_j) \quad , \quad \forall a_j \neq a \end{aligned}$$

Resolución mediante ascenso de gradiente

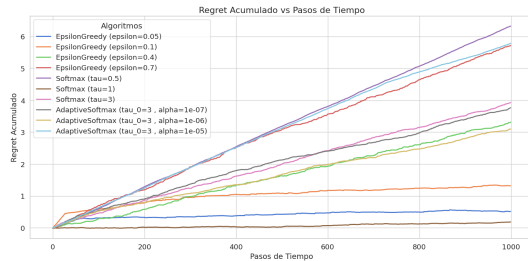
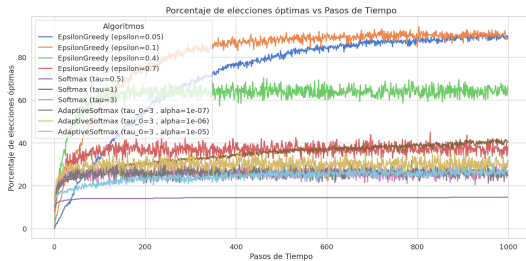
Experimento con brazos de distribución Normal

Softmax VS Gradiente de preferencias



Resolución mediante ascenso de gradiente

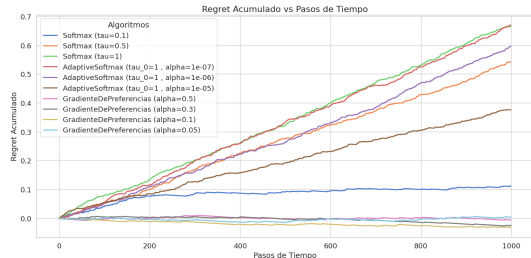
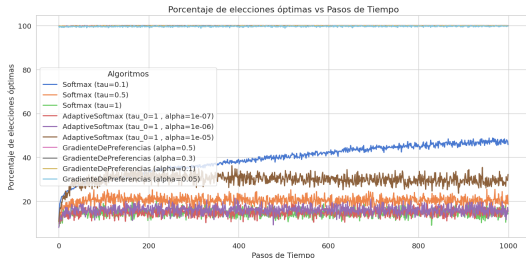
Softmax VS ϵ -greedy



Resolución mediante ascenso de gradiente

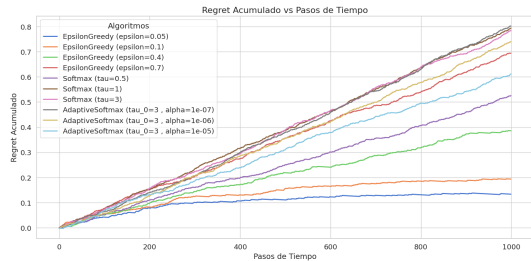
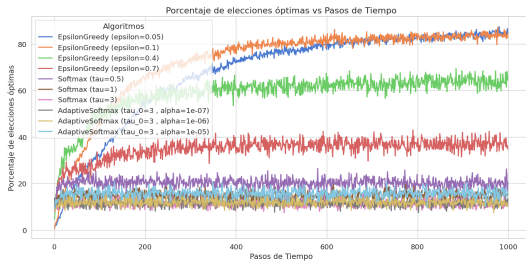
Experimento con brazos de distribución Bernoulli

Softmax VS Gradiente de preferencias



Resolución mediante ascenso de gradiente

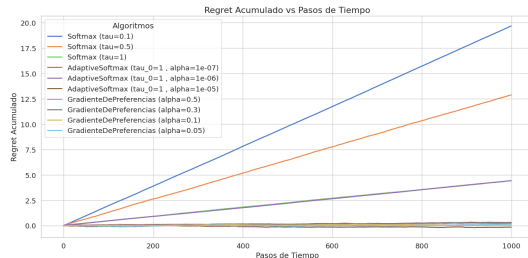
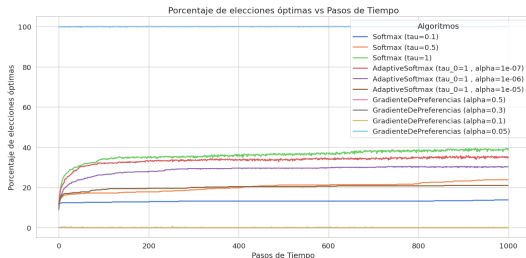
Softmax VS ϵ -greedy



Resolución mediante ascenso de gradiente

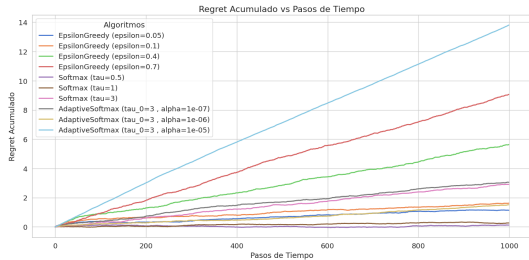
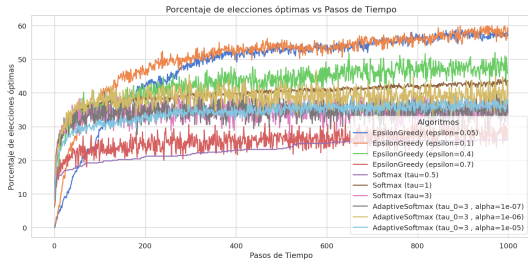
Experimento con brazos de distribución Binomial

Softmax VS Gradiente de preferencias



Resolución mediante ascenso de gradiente

Softmax VS ϵ -greedy



Resolución mediante ascenso de gradiente

Conclusiones

- ▶ La estrategia más simple no es necesariamente la peor
- ▶ Algunos algoritmos "no consiguen aprender"
- ▶ El desempeño de los algoritmos varía al cambiar la distribución de los brazos del bandido
- ▶ Podemos elegir entre desempeño a medio plazo (regret acumulado) y a largo plazo (porcentaje de elecciones de brazo óptimo)

Resolución mediante ascenso de gradiente

Top de mejores modelos a medio y largo plazo

A medio plazo

► Con distribución Normal

1. **Cualquier GDP**
2. $\text{Softmax}(\tau = 1)$
3. $\epsilon\text{-greedy}(\epsilon = 0,05)$

► Con distribución Bernoulli

1. **Cualquier GDP**
2. $\text{Softmax}(\tau = 0,1)$
3. $\epsilon\text{-greedy}(\epsilon = 0,05)$

► Con distribución Binomial

1. **Cualquier GDP**
2. $\epsilon\text{-greedy}(\epsilon = 0,05)$
3. Adaptive Softmax ($\tau_0 = 3, \alpha = 1^{-7}$)

A largo plazo

► Con distribución Normal

1. **Cualquier GDP**
2. $\epsilon\text{-greedy}(\epsilon = 0,05)$ (90 %)
3. $\epsilon\text{-greedy}(\epsilon = 0,1)$ (90 %)

► Con distribución Bernoulli

1. **Cualquier GDP**
2. $\epsilon\text{-greedy}(\epsilon = 0,05)$ (80 %)
3. $\epsilon\text{-greedy}(\epsilon = 0,01)$ (80 %)

► Con distribución Binomial

1. **Cualquier GDP**
2. $\epsilon\text{-greedy}(\epsilon = 0,05)$ (60 %)
3. $\epsilon\text{-greedy}(\epsilon = 0,1)$ (60 %)

The background features two large, overlapping triangular shapes. The one on the left is a teal color, and the one on the right is a dark blue color. They meet at a point in the center, creating a white triangular area where the text is located.

Resolución mediante algoritmos UCB

Resolución mediante algoritmos UCB

Recordatorio

- ▶ El método UCB mide el potencial de cada brazo a mediante:

$$ucb(a) = Q(a) + u(a)$$

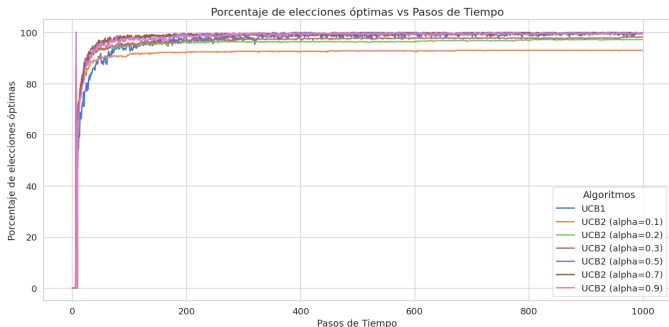
- ▶ Se selecciona la acción que maximiza el límite superior de confianza:

$$a_t = \arg \max_{a \in A} ucb(a)$$

- ▶ Valor $UCB1(a) = Q(a) + \sqrt{\frac{2 \ln t}{N(a)}}$
- ▶ Valor $UCB2(a) = Q(a) + \sqrt{\frac{(1+\alpha) \ln\left(\frac{t}{\tau(k_a)}\right)}{2\tau(k_a)}}$
 - ▶ $0 < \alpha < 1$: parámetro de ajuste del balance entre exploración y explotación.
 - ▶ k_a : número de épocas de la acción a .
 - ▶ $\tau(k_a) = \lceil (1 + \alpha)^{k_a} \rceil$: número de veces que se seleccionará a en una época.

Resolución mediante algoritmos UCB

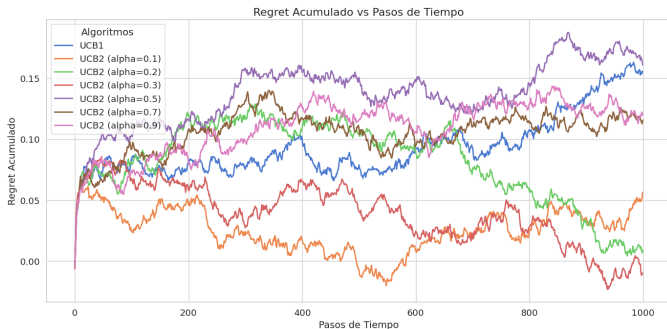
UCB1 vs UCB2 - Distribución Normal



- ▶ **UCB1:** Crecimiento lento al inicio, pero alcanza valores elevados a largo plazo.
- ▶ **UCB2 ($\alpha = 0,1, 0,2, 0,3$):** Crecen rápido, pero convergen en valores inferiores a UCB1 por explotación temprana.
- ▶ **UCB2 ($\alpha = 0,5$):** Equilibrio eficiente entre exploración y explotación, superando ligeramente a UCB1.
- ▶ **UCB2 ($\alpha = 0,7, 0,9$):** Rendimiento similar a $\alpha = 0,5$, pero algo menor por exploración prolongada.

Resolución mediante algoritmos UCB

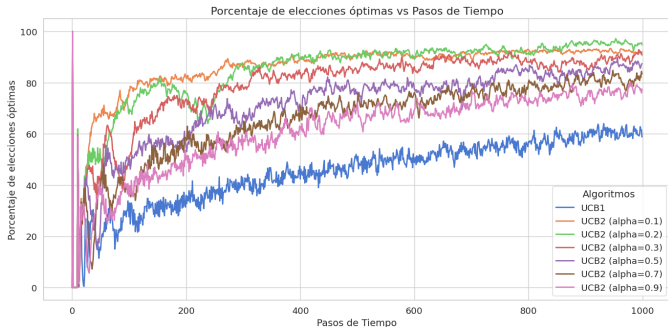
UCB1 vs UCB2 - Distribución Normal



- ▶ **UCB1, UCB2 ($\alpha = 0,5, 0,7, 0,9$):** Alta exploración, lo que aumenta el regret.
- ▶ **UCB2 ($\alpha = 0,1, 0,2, 0,3$):** Regret crece al inicio, pero se estabiliza, mayor explotación.
- ▶ Todos los algoritmos UCB logran un regret acumulado bajo ($< 0,2$).
- ▶ Los valores más bajos de α son más eficientes para minimizar el regret.

Resolución mediante algoritmos UCB

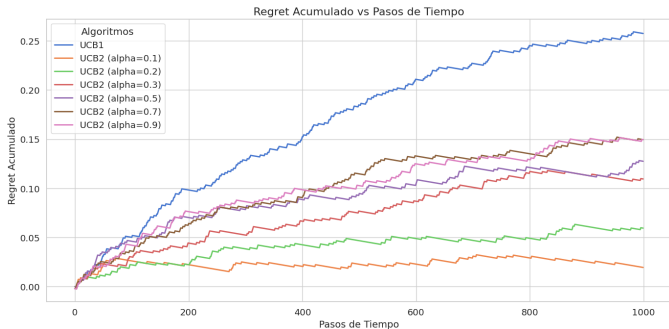
UCB1 vs UCB2 - Distribución Bernoulli



- **UCB1:** Peores resultados, crecimiento lento y valores finales bajos.
- **UCB2:** Mejores resultados con α más pequeño ($\alpha = 0,1, 0,2$), con Bernoulli es más favorable la explotación temprana.

Resolución mediante algoritmos UCB

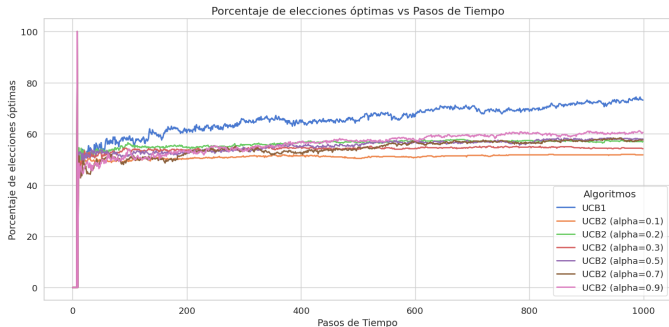
UCB1 vs UCB2 - Distribución Bernoulli



- ▶ **UCB1:** Mayor regret acumulado, exploración-explotación menos eficiente.
- ▶ **UCB2 ($\alpha = 0,3, 0,5, 0,7, 0,9$):** Mejor que UCB1, pero el regret aumenta por exceso de exploración.
- ▶ **UCB2 ($\alpha = 0,1, 0,2$):** Regret muy bajo, especialmente $\alpha = 0,1$, por rápida identificación y explotación del brazo óptimo.

Resolución mediante algoritmos UCB

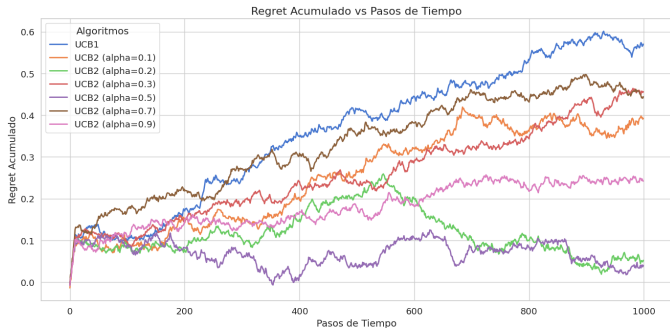
UCB1 vs UCB2 - Distribución Binomial



- **UCB1:** Mayor porcentaje de elecciones óptimas.
- **UCB2:** Rendimiento inferior a UCB1. A medida que α aumenta, mejora ligeramente el porcentaje de elecciones óptimas por mayor exploración inicial.

Resolución mediante algoritmos UCB

UCB1 vs UCB2 - Distribución Binomial



- **UCB1:** Mayor regret acumulado por una estrategia más rígida.
- **UCB2:** Mejor que UCB1, destacan $\alpha = 0,2, 0,5$, aunque podría deberse a la semilla seleccionada.

Resolución mediante algoritmos UCB

UCB1 vs UCB2 - Conclusión final

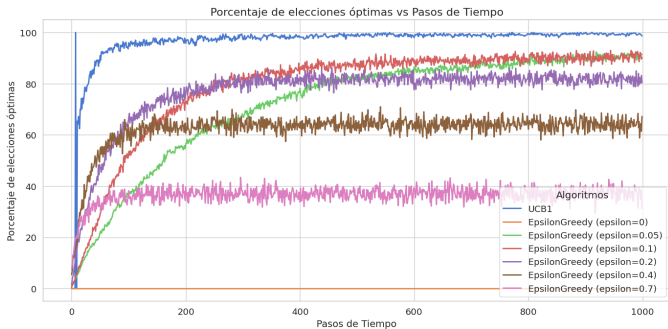
Distribución	Porcentaje de elecciones óptimas	Regret acumulado
Normal	UCB2 ($\alpha = 0,7$), UCB2 ($\alpha = 0,5$)	UCB2 ($\alpha = 0,3$), UCB2 ($\alpha = 0,2$)
Bernoulli	UCB2 ($\alpha = 0,2$), UCB2 ($\alpha = 0,1$)	UCB2 ($\alpha = 0,1$), UCB2 ($\alpha = 0,2$)
Binomial	UCB1, UCB2 ($\alpha = 0,9$)	UCB2 ($\alpha = 0,5$), UCB2 ($\alpha = 0,2$)

Cuadro 1: Mejores resultados según el estadístico y la distribución de los brazos

- El algoritmo que ofrece mejor desempeño cambia al modificar la distribución de los brazos del bandido.

Resolución mediante algoritmos UCB

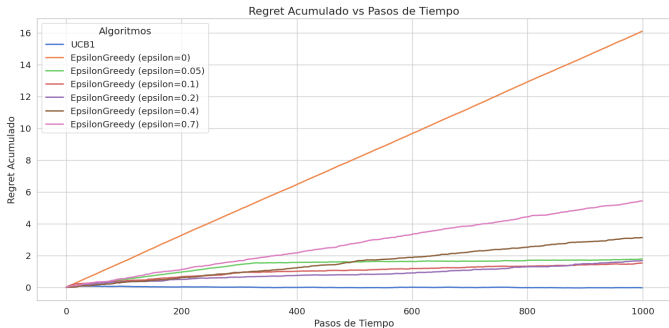
ϵ -greedy vs UCB1 - Distribución Normal



- ▶ **UCB1:** Crecimiento rápido, alcanza valores cercanos al 100 % de elecciones óptimas rápidamente, mejor desempeño.
- ▶ **ϵ -greedy ($\epsilon = 0,05, 0,1, 0,2$):** Buen rendimiento, pero inferior a UCB1.
- ▶ **ϵ -greedy ($\epsilon = 0,4, 0,7$):** Excesiva exploración, estancamiento rápido en un valor bajo con mayor oscilación.
- ▶ **ϵ -greedy ($\epsilon = 0$):** Sin exploración, 0 % de elecciones óptimas por mala elección inicial.

Resolución mediante algoritmos UCB

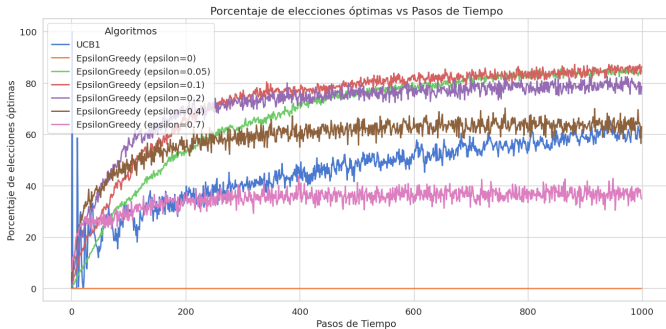
ϵ -greedy vs UCB1 - Distribución Normal



- ▶ **UCB1:** Regret acumulado cercano a 0, por su balance entre exploración y explotación.
- ▶ **ϵ -greedy ($\epsilon = 0,05, 0,1, 0,2$):** Buen desempeño, descubren rápidamente el brazo óptimo.
- ▶ **ϵ -greedy ($\epsilon = 0,4, 0,7$):** Resultados algo peores por exceso de exploración.
- ▶ **ϵ -greedy ($\epsilon = 0$):** Regret crece linealmente y muy alto, sin exploración.

Resolución mediante algoritmos UCB

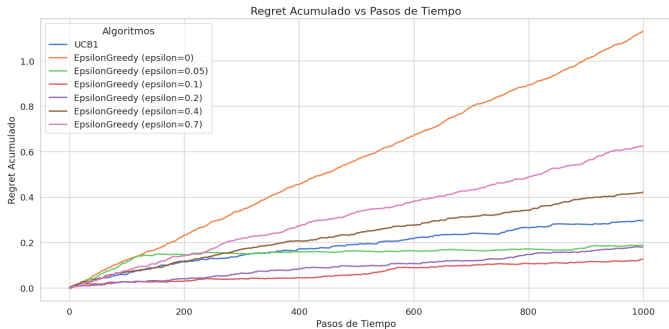
ϵ -greedy vs UCB1 - Distribución Bernoulli



- ▶ **ϵ -greedy ($\epsilon = 0,05, 0,1, 0,2, 0,4$):** Mejores que UCB1.
- ▶ **UCB1:** Crece más lento y alcanza un peor porcentaje que muchos ϵ -greedy.
- ▶ **ϵ -greedy ($\epsilon = 0,7$):** Peor que UCB1 por excesiva exploración.
- ▶ **ϵ -greedy ($\epsilon = 0$):** Sin exploración, 0% de elecciones óptimas.

Resolución mediante algoritmos UCB

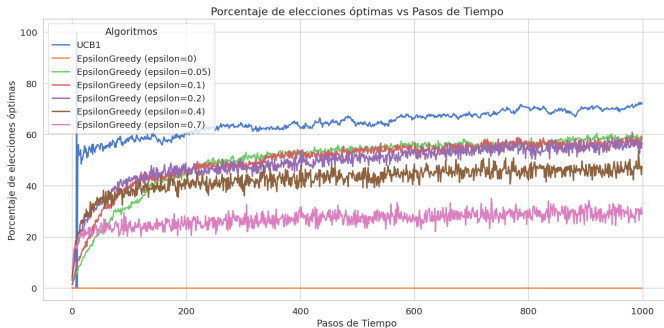
ϵ -greedy vs UCB1 - Distribución Bernoulli



- ▶ ϵ -greedy ($\epsilon = 0,05, 0,1, 0,2$): Buen equilibrio, mejores que UCB1.
- ▶ UCB1: Desempeño intermedio, con regret acumulado bajo.
- ▶ ϵ -greedy ($\epsilon = 0,4, 0,7$): Excesiva exploración, peores que UCB1.
- ▶ ϵ -greedy ($\epsilon = 0$): Peor desempeño, regret creciente por falta de exploración.

Resolución mediante algoritmos UCB

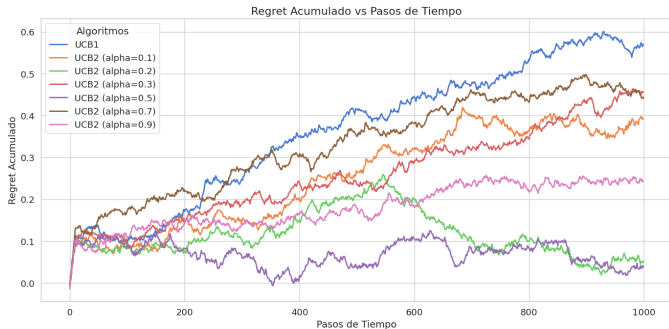
ϵ -greedy vs UCB1 - Distribución Binomial



- ▶ **UCB1:** Mejor desempeño, con crecimiento rápido y altos valores de elecciones óptimas.
- ▶ **ϵ -greedy ($\epsilon = 0,05, 0,1, 0,2$):** Buen desempeño, superan al resto de los algoritmos ϵ -greedy.
- ▶ **ϵ -greedy ($\epsilon = 0,4, 0,7$):** Peores por excesiva exploración.
- ▶ **ϵ -greedy ($\epsilon = 0$):** Peor desempeño, atrapado en una mala elección inicial.

Resolución mediante algoritmos UCB

ϵ -greedy vs UCB1 - Distribución Binomial



- ▶ **UCB1:** Mejor desempeño con un regret bajo, gracias al balance eficaz entre exploración y explotación.
- ▶ **ϵ -greedy ($\epsilon = 0,05, 0,1, 0,2$):** El regret se estabiliza rápidamente y permanece bajo.
- ▶ **ϵ -greedy ($\epsilon = 0,4, 0,7$):** Desempeño más pobre, especialmente $\epsilon = 0,7$, que explora en exceso.
- ▶ **ϵ -greedy ($\epsilon = 0$):** Peor desempeño, regret creciente por falta de exploración.

Resolución mediante algoritmos UCB

ϵ -greedy vs UCB1 - Conclusión final

Distribución	Porcentaje de elecciones óptimas	Regret acumulado
Normal	UCB1, ϵ -greedy ($\epsilon = 0,1$)	UCB1, ϵ -greedy ($\epsilon = 0,1$)
Bernoulli	ϵ -greedy ($\epsilon = 0,1, \epsilon = 0,05$)	ϵ -greedy ($\epsilon = 0,1, \epsilon = 0,2$)
Binomial	UCB1, ϵ -greedy ($\epsilon = 0,05$)	UCB1, ϵ -greedy ($\epsilon = 0,1$)

Cuadro 2: Mejores resultados según el estadístico y la distribución de los brazos

- ▶ Destacan UCB1 y ϵ -greedy con valores bajos de ϵ .
- ▶ La falta completa de exploración (ϵ -greedy con $\epsilon = 0$) hace que una primera mala elección perjudique completamente el rendimiento.
- ▶ Un exceso de exploración (valores elevados de ϵ) también es perjudicial.

Resolución mediante algoritmos UCB

ϵ -greedy vs UCB2 - Conclusión final

Distribución	Porcentaje de elecciones óptimas	Regret acumulado
Normal	UCB2 ($\alpha = 0,5$), UCB2 ($\alpha = 0,9$)	UCB2 ($\alpha = 0,1$), UCB2 ($\alpha = 0,5$)
Bernoulli	UCB2 ($\alpha = 0,1$), UCB2 ($\alpha = 0,3$)	UCB2 ($\alpha = 0,1$), UCB2 ($\alpha = 0,3$)
Binomial	UCB2 ($\alpha = 0,9$), ϵ -greedy ($\epsilon = 0,05$)	UCB2 ($\alpha = 0,5$), UCB2 ($\alpha = 0,9$)

Cuadro 3: Mejores resultados según el estadístico y la distribución de los brazos

- ▶ UCB2 supera a ϵ -greedy en la mayoría de los casos.
- ▶ Bernoulli es discreta (0 o 1), causando fluctuaciones iniciales que pueden llevar a explorar en exceso brazos subóptimos. UCB2 con valores bajos de α es más efectivo, pues favorece explotación.
- ▶ Normal y Binomial toman más valores, UCB2 con valores altos de α es más efectivo.

The background features two large, overlapping geometric shapes. On the left, a teal-colored triangle points downwards. On the right, a dark blue triangle points upwards. They meet at a central point, creating a white triangular area where the text is located.

Introducción a problemas más complejos

Reinforcement Learning(RL)

Implica entrenar a un agente para que aprenda a tomar decisiones secuenciales en un entorno con el objetivo de maximizar una señal de recompensa a largo plazo.

A diferencia de otros enfoques como el aprendizaje supervisado o no supervisado, el agente en RL aprende mediante prueba y error, recibiendo retroalimentación en forma de recompensas o castigos, lo que le permite aprender de sus propias acciones y mejorar con el tiempo

Componentes clave del aprendizaje por refuerzo

- ▶ **Agente**
- ▶ **Entorno**
- ▶ **Estado**
- ▶ **Acción**
- ▶ **Recompensa**
- ▶ **Política**
- ▶ **Valor**

Proceso de aprendizaje

- ▶ El agente comienza en un estado inicial del entorno
- ▶ Elige una acción según su política actual.
- ▶ El entorno responde, cambiando de estado y otorgando una recompensa (o penalización) al agente.
- ▶ El agente usa esta recompensa para ajustar su política con el fin de maximizar futuras recompensas.

Este proceso de interacción, retroalimentación y ajuste de políticas es lo que permite al agente mejorar con el tiempo

Frozen Lake

El entorno Frozen Lake consiste en un lago congelado representado por una cuadrícula de celdas. El objetivo del agente es navegar a través de esta cuadrícula sin caer en los agujeros (huecos en el hielo) y llegar a la casilla de meta (objetivo) de la cuadrícula.

Acciones El agente tiene cuatro posibles acciones las cuales dependen del estado donde se encuentre:

- ▶ **Izquierda (Left):** Moverse a la celda a la izquierda
- ▶ **Derecha (Right):** Moverse a la celda a la derecha
- ▶ **Arriba (Up):** Moverse a la celda hacia arriba
- ▶ **Abajo (Down):** Moverse a la celda hacia abajo



Resolución de Frozen Lake con Monte Carlo On-policy

Resolución de Frozen Lake

con Monte Carlo On-policy

Implementación: Monte Carlo On-policy all visits

Sea π una política epsilon-soft, $Q(s, a) \in \mathbb{R}$ arbitrario y $Returns(s, a)$ una lista vacía.

Se repite para cada episodio:

Se genera un episodio completo siguiendo π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$
 $G = 0$

Para cada paso del episodio, $t = T - 1, T - 2, \dots, 0$:

$$G = \gamma G + R_{t+1}$$

$$Returns(S_t, A_t).append(G)$$

$$Q(S_t, A_t) = average(Returns(S_t, A_t))$$

$$A^* = arg \max_a Q(S_t, a)$$

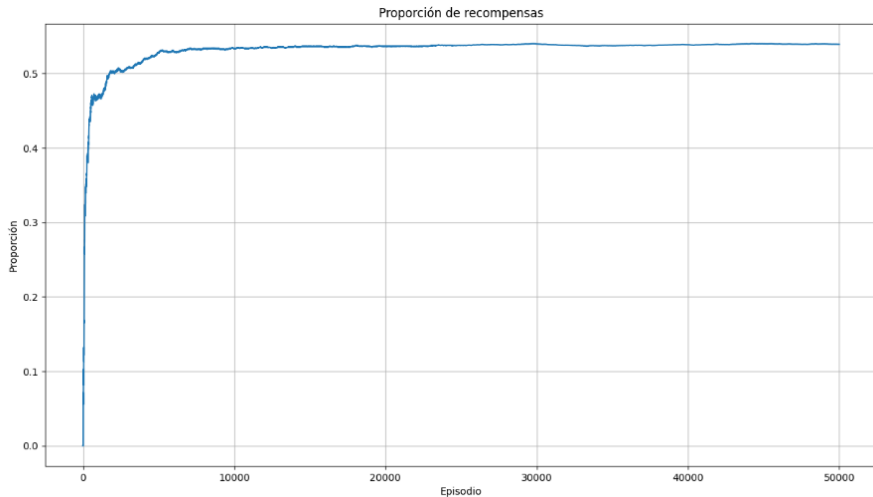
Resolución de Frozen Lake con Monte Carlo On-policy

Frozen Lake 4x4



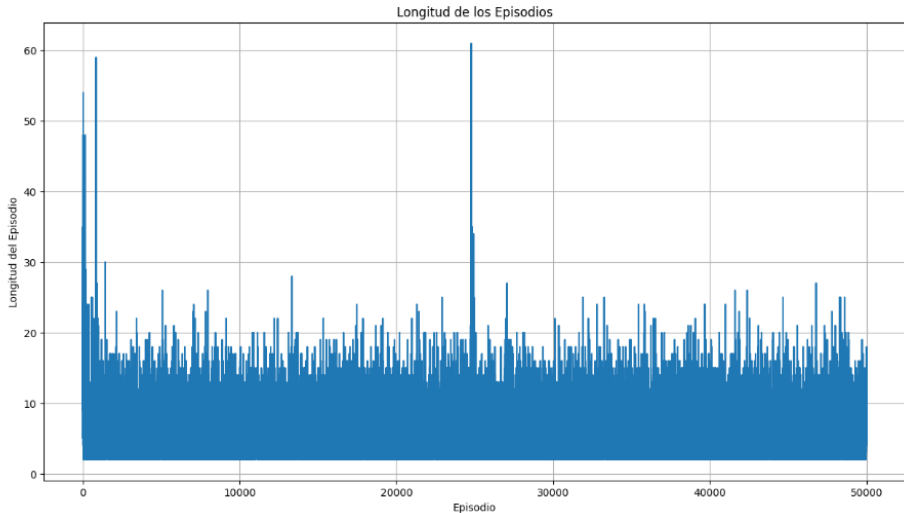
- ▶ Consiste en cruzar un lago helado desde *Start*(*S*) hasta *Goal*(*G*) sin caer en ningún agujero.
- ▶ Se utiliza la versión no resbaladiza.
- ▶ Acciones posibles:
 - ▶ 0: LEFT
 - ▶ 1: DOWN
 - ▶ 2: RIGHT
 - ▶ 3: UP
- ▶ Recompensas:
 - ▶ Llegar a Goal(*G*): +1
 - ▶ Llegar a Hole(*H*): 0
 - ▶ Llegar a Frozen(*F*): 0

Resolución de Frozen Lake con MC On-policy

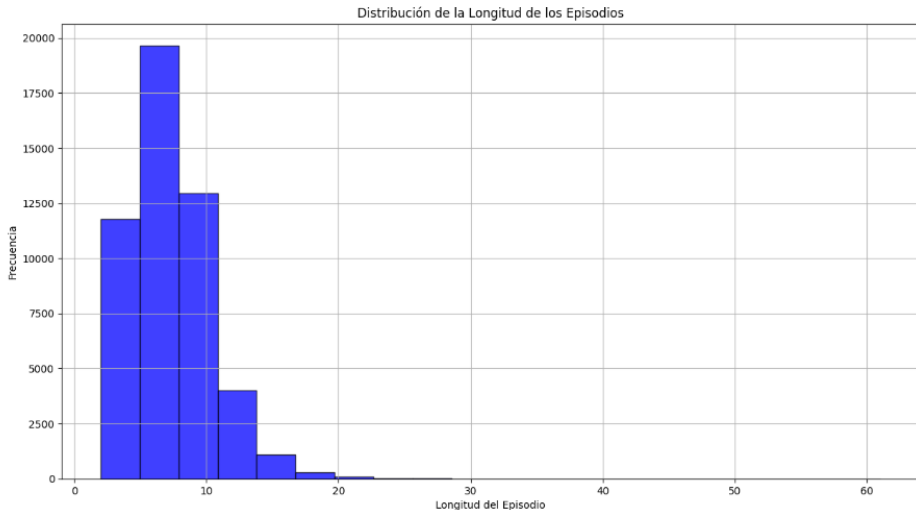


Máxima proporción: 0.53914

Resolución de Frozen Lake con MC On-policy



Resolución de Frozen Lake con MC On-policy



Resolución de Frozen Lake con MC On-policy

Política óptima

Valores Q para cada estado:

```
[[0.5233557 0.54184208 0.51488692 0.52784965]
 [0.52352941 0.          0.59408916 0.4723779 ]
 [0.52659574 0.61745442 0.49880383 0.61704312]
 [0.57756233 0.          0.35294118 0.35057471]
 [0.52964297 0.62144847 0.          0.53638089]
 [0.          0.          0.          0.          ]
 [0.          0.80045967 0.          0.5952381 ]
 [0.          0.          0.          0.          ]
 [0.61967938 0.          0.72026683 0.54694033]
 [0.62969082 0.82535984 0.81039755 0.          ]
 [0.7348659 0.96039276 0.          0.62186235]
 [0.          0.          0.          0.          ]
 [0.          0.          0.          0.          ]
 [0.          0.83428416 0.96027136 0.72796518]
 [0.82530738 0.9557061 1.          0.81931464]
 [0.          0.          0.          0.          ]]
```

Política greedy: $\pi'(s) = \arg \max_a Q(s, a)$

Política óptima obtenida

```
[[0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 1. 0. 0.]
 [1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [1. 0. 0. 0.]
 [0. 0. 1. 0.]
 [0. 1. 0. 0.]
 [0. 1. 0. 0.]
 [1. 0. 0. 0.]
 [1. 0. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 1. 0.]
 [1. 0. 0. 0.]]
```

Acciones [1, 1, 2, 1, 2, 2]

Resolución de Frozen Lake con MC On-policy

Política óptima

- ▶ Acciones: [1, 1, 2, 1, 2, 2]
- ▶ Acciones posibles:
 - ▶ 0: LEFT
 - ▶ 1: DOWN
 - ▶ 2: RIGHT
 - ▶ 3: UP



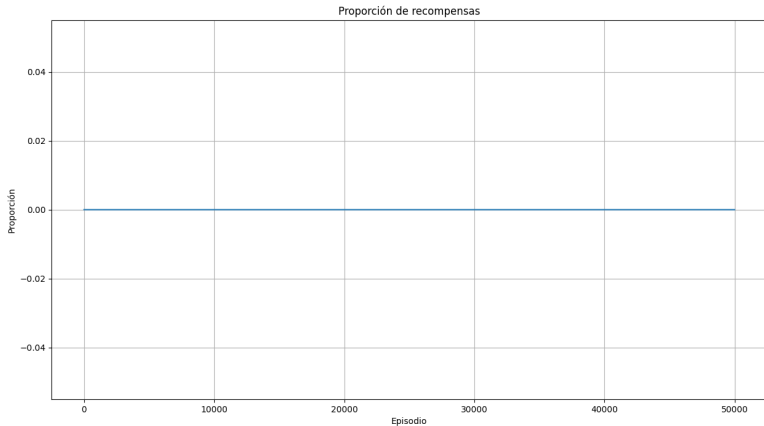
Propuestas de mejora

Propuestas de mejora

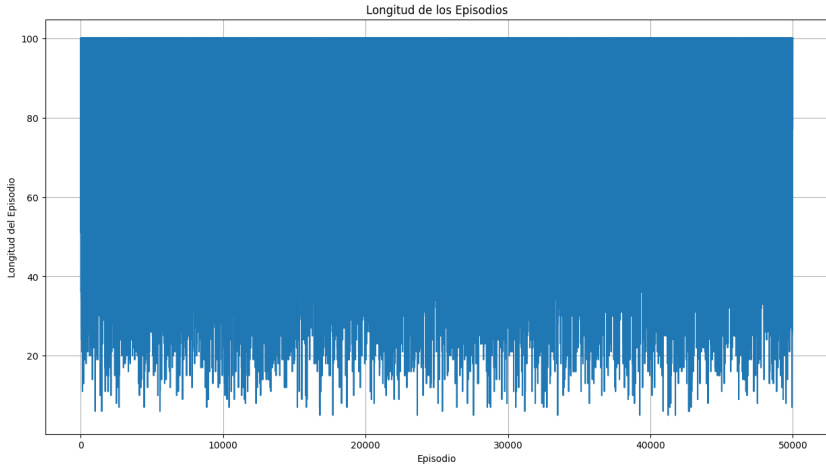
Fracaso en el escenario 8×8



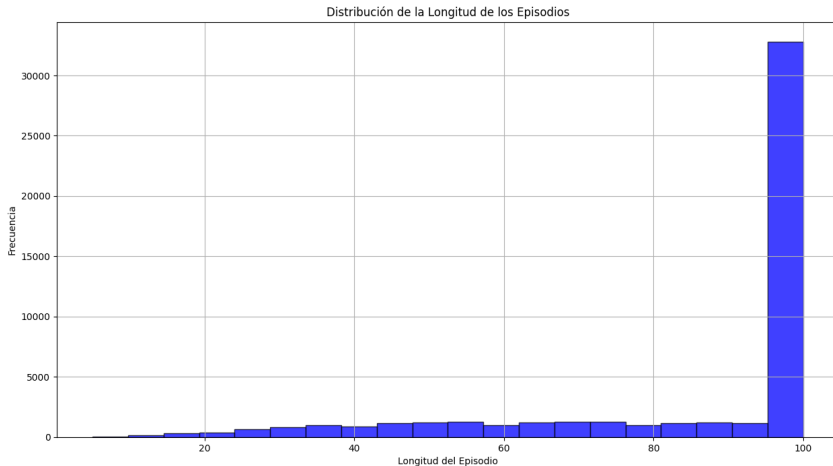
Propuestas de mejora



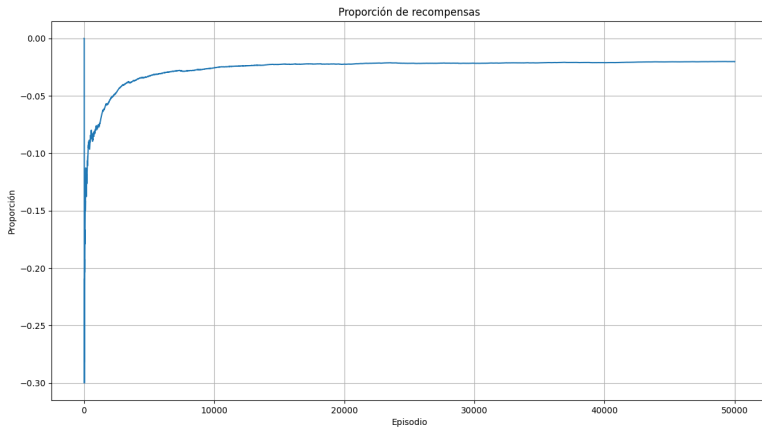
Propuestas de mejora



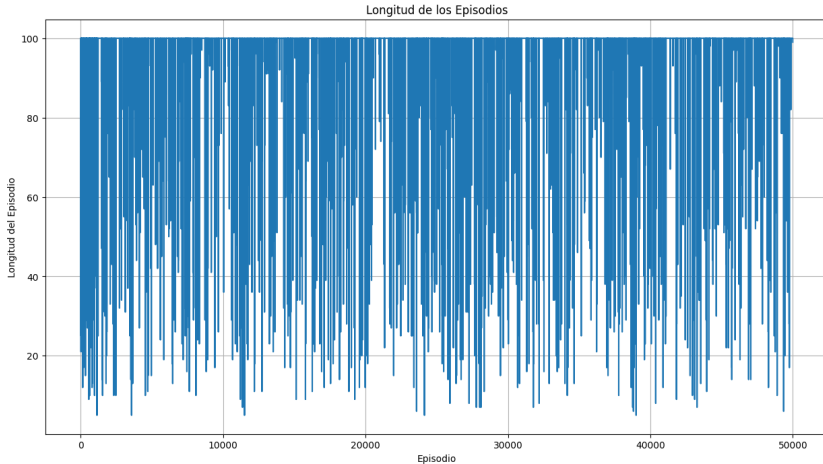
Propuestas de mejora



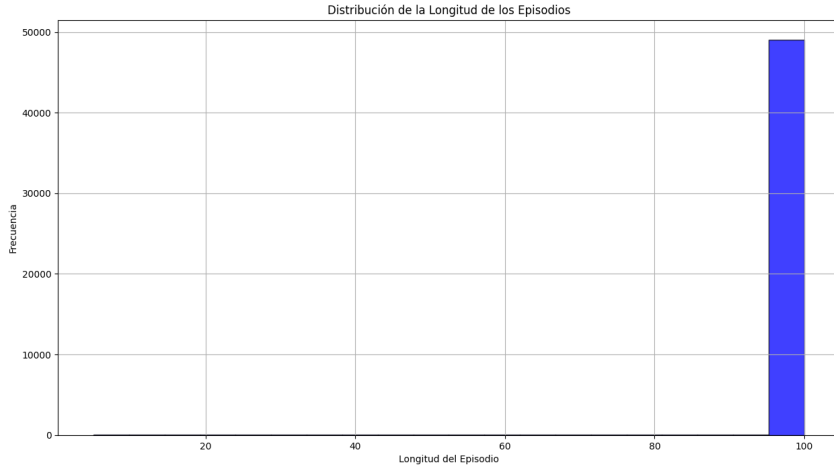
Propuestas de mejora



Propuestas de mejora



Propuestas de mejora



Propuestas de mejora

Conclusiones

- ▶ **Problema:** Montecarlo On-policy all visits **no consigue aprender** la ruta en 8×8
Solución: Aumentar el límite de pasos por episodio y/o cambiar el sistema de recompensas del entorno
- ▶ **Problema:** El algoritmo solo busca una ruta factible, pero **no optimiza la ruta**
Solución: Cambiar el sistema de recompensas para que cada desplazamiento tenga un coste asociado
- ▶ **Problema:** La **naturaleza** de Monte Carlo es **desfavorable** para resolver el problema
Solución: Implementar agentes más robustos basados en diferencias temporales (SARSA, Q-learning)

